

Nested queries in DML operations

1. update smiths salary to allen's salary

update emp

set sal=(select sal from (select * from emp) e where e.ename='allen')
where ename='SMITH';

2. delete all rows of employees who works in same department as allen

delete from emp

where deptno=(select deptno from (select * from emp) e
where e.ename='Allen');

3. create a copy of emp table

create table emp1

as

(select * from emp);

1. create a copy of emp table but copy only empno, name and deptno

Create table emp2

As

(select empno,ename,deptno
From emp)

2. create a copy of emp table but copy only empno, name and deptno, do not copy any data

Create table emp2

As

(select empno,ename,deptno
From emp where 1=2
)

3. insert data of all employees from department 20, in emp2 table

insert into emp2

(select * from emp
Where deptno=20)

1. Find first 3 topmost employees

1.	Rajan.	50000	1 → 0 ✓
2.	Ashwin.	30000	2 — 1 ✓
3.	Sanjay.	50000	1 — 0 ✓
4.	Deepa.	30000	2 — 1 ✓
5.	Ashu.	20000	4 — 3 ✗
6.	Sangeeta.	25000	3 — 2 ✗
7.	Kedar.	50000	1 — 0 ✓

```
select e.empno,e.ename,e.sal  
from emp e  
where 2> (select count(*)  
from emp m  
where m.sal > e.sal;
```

```
select * from emp e  
where 3 > (select count(*) from (select distinct sal from emp) m
```

```
where m.sal>e.sal)
order by sal desc;
```

1. Find 3rd topmost employees

```
select * from emp m
  where 2=(select count(*) from (select distinct sal from emp) e
  where e.sal > m.sal)
order by sal desc;
```

window functions

all aggregate functions are window functions

```
select empno,ename,deptno,avg(sal) over (partition by deptno), sum(sal) over (partition by
deptno)
```

```
from emp;
```

row_number()	assign unique value to every row within window <pre>select empno,ename,sal,deptno,row_number() over (partition by deptno order by sal desc) from emp;</pre>
rank()	assign number to distinct values, if the values are same then same rank will be assigned to both rows. but when more than one row gets same rank then it will skip numbers and then the next rank will be assigned <pre>select empno,ename,sal,deptno,rank() over (partition by deptno order by sal) from emp;</pre> to get bottommost 3 salaries <pre>select empno, ename,sal,deptno,rnk from (select empno,ename,sal,deptno,rnk k() over (order by sal) rn from emp) nemp where rn<=3</pre> or <pre>with rank_tab as (select empno,ename,sal,deptno,rnk k() over (order by sal) rn from emp)</pre> <pre>select empno, ename,sal,deptno,rnk from rank_tab r where rn<=3;</pre>
dense_rank()	assign number to distinct values, if the values same then same dense rank will be assigned to both rows. but when more than one row gets

	same dense rank, then it will not skip in between numbers and then the next rank will be assigned to get bottommost 3 salaries <pre>select empno, ename,sal,deptno,rnk from (select empno,ename,sal,deptno,dense_rank() over (order by sal) drnk from emp) nemp where drnk<=3</pre> or <pre>with rank_tab as (select empno,ename,sal,deptno,dense_rank() over (order by sal) drnk from emp)</pre> <pre>select empno, ename,sal,deptno,drnk from rank_tab r where drnk<=3;</pre> ---- to find department wise bottom most <pre>select empno,ename,sal,deptno,drnk from(select empno,ename,sal,deptno,dense_rank() over (partition by deptno order by sal desc) drnk from emp) nemp where drnk=1;</pre>
lag(val, n)	it will find nth previous value
lead(val,n)	it will find nth next value
first_value(sal)	it will find first value of the given column within window
last_value(sal)	it will find last value of the given column within window <pre>SELECT empno, ename, sal, LAST_VALUE(ename) OVER (PARTITION BY deptno ORDER BY sal ASC ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS highest_paid_employee FROM emp;</pre> Note : UNBOUNDED PRECEDING Means: Start from the first row of the department. UNBOUNDED FOLLOWING Means: Go all the way to the last row of the department.

Frame

			first_value.	last value	lag(sal,1)	lead(sal,1)
10 Ashu	10000	100	10000	11000	null	11000
11. Deepa	11000	100	10000	11000	10000	null
12 Sana.	12000	200	12000	20000	null	13000
13. raj.	13000	200	12000	20000	12000	150000
14 rr.	15000	200	12000	20000	13000	20000
15. aradhya	20000	200	12000	20000	15000	null

- Find difference between employee sal and the topmost salary of each department

```
select empno,ename, deptno,sal,topsal,topsal-sal difference
from (select empno,ename,deptno,sal,first_value(sal) over (partition by deptno order
by sal desc) topsal from emp) nemp;
```

or

```
with nemp as (select empno,ename,deptno,sal,first_value(sal) over (partition by
deptno order by sal desc) topsal from emp)
select empno,ename, deptno,sal,topsal,topsal-sal difference
from nemp e;
```

- Find cummulative sum of sal

```
SELECT empno,ename, sal, SUM(sal) OVER (ORDER BY empno) AS
cumulative_sal
FROM emp;
```

In the following example partition by will divide the data into window and order by will arrange data within window

```
select deptno,ename,sum(sal) over (partition by deptno)
from emp
```

```
select empno,ename,sal,deptno,row_number() over (),rank() over (order by sal desc)
from emp;
```

- to find highly paid employee

```
select * from (
```

```
select empno,ename,sal,deptno,row_number() over () rownum,rank() over (order by sal
desc) rn,dense_rank() over (order by sal desc) drn from emp) e
```

```
where e.drn=1;
```

- to find highly paid employee in each department

```
select * from (
```

```
select empno,ename,sal,deptno,row_number() over () rownum,rank() over (order by sal
desc) rn,dense_rank() over (partition by deptno order by sal desc) drn from emp) e
```

```
where e.drn=1;
```

- find expensive product in each category

```
with nprod as (select pid,pname,catid,price,dense_rank() over (partition by catid order
by price desc) drnk
from product)
select * from nprod e where drnk=1;
```

4. to find first sal in each window

```
select empno,ename,sal,deptno,first_value(sal) over (partition by deptno
order by sal) fv
from emp;
```

5. add next salary into current sal

```
with nemp as (select empno,ename,sal,deptno,sal,lead(sal,1) over () lagval
from emp)
select * from nemp e;
```

indexes

2 types of indexes

1. clustered index

- a. there will be only one clustered index
- b. it does not require extra space because it is stored along with data in the table

2. non clustered index

- a. there can be many non clustered index
- b. these are stored outside table, and hence need extra space

indexes speed up the search action , but reduces the speed of DML statements, so donot create unnecessary indexes.

Types of indexes

1. primary key index-----indexes on primary key gets create automatically
2. unique index
 - a. indexes on column, with unique constraint gets created automatically
 - b. the fields on which unique index is created, then duplicate values are not allowed in that column.

```
create unique index idx_passport
on emp(password desc)
```

3. regular index

```
create index idx_sal on emp(sal)
```

4. full text indexes

- a. These are usually used on text type column,
- b. it stores phrases or words and their position

```
create full text index idx_profile
```

```
on emp(profile)
```

5. geospatial index – It is used when the field store geographical location.

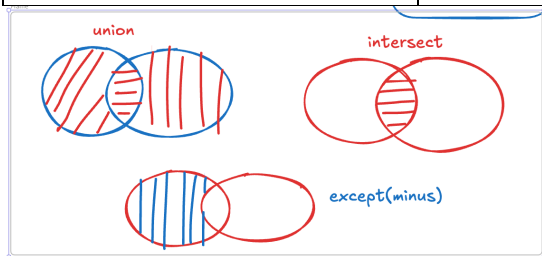
to drop the index

drop index `idx_passport` on emp

show indexes from emp;

set operator

Union	It will display all common rows of multiple query only once, and also display rows which are only in 1 st query and rows only in 2 nd query
Union all	It will display all common rows of multiple query multiple times, and also display rows which are only in 1 st query and rows only in 2 nd query
Intersect	Will display only common rows
Except (minus)	It will display rows which are only in 1 st query



Views in mysql

There are 2 types views in database

1. view
2. materialized view---- when you are working on static data, then it is good to create materialized view.
Once you fire the query 1st time data will be retrieved, it will get save in RAM for the current session.
It speeds up the job of retrieval of the data.

for every view, separate table will not get created, only base query gets stored for view.

if we fire select statement on view, then the base query will get executed

while creating view, if we use with check option, if view based on single table, and if view contains all not null columns, then one can perform DML operations on the view. and only valid data for which where condition is satisfied, can be added or removed or updated

To stop all DML operations on the view, use with read only option

why to use views

1. to give limited access to the table
2. hide complex queries under the view
3. we may hide table names, which increases security.

to create view

create <materialized> view < name of the view>

as

<base query>

to create a view for manager of dept 10 to give access to all the records of dept 10

create view mgr10

as

(select * from emp

where deptno=10

with check option). With readonly option (but readonly does not work in mysql)

create view testview1

-> as

-> select empno,ename,e.deptno edeptno,d.deptno ddeptno,d.dname

-> from emp e, dept d

-> where e.deptno=d.deptno;

create view testview

-> as

-> select deptno,job,sum(sal+ifnull(comm,0)),count(*)

-> from emp

-> group by deptno;